

rvm_log_fileline.vri

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/log/rvm_log_fileline.vri

```
//-----
// This confidential and proprietary software may be used only as authorized
// by a licensing agreement from Synopsys Inc. In the event of publication,
// the following notice is applicable:
//
// (C) COPYRIGHT 2004 SYNOPSYS INC. ALL RIGHTS RESERVED
//
// The entire notice above must be reproduced on all authorized copies.
//-----
// Filename      : $Id: rvm_log_fileline.vri,v 1.1.2.3 2004/09/03 16:06:02 janick Exp $
//
// Created by    : Chris Spear, Synopsys Inc. 03/09/2004
//                $Author: janick $
//
// Description   : Redefines the macros used with rvm_log to print the file name
//                and number before the message.
//
// Usage        : Include after <rvm_std_lib.vrh>. See "xactor_base_log.vr" for
//                an example.
//
//                rvm_warning("This is a warning");
//
//                will produce (using the default format):
//
//                WARNING [Failure] on program(none) at 0 ns:
//                "sample.vr", 18: This is a warning
//
// Notes        : Using these macros will increase the run-time required
//                to issue all messages.
//
//                Format can be modified as needed.
//                Interesting compile-time variables include __DATE__, __FILE__,
//                __LINE__, __TIME__, __VERA__, __VERA_MINOR__, __VERA_PATCH__,
//                __VERA_VERSION__, and __VERSION__
//-----
```

```

#undef rvm_warning
#define rvm_warning(log, msg) \
if (log.start_msg(log.FAILURE_TYP, log.WARNING_SEV)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_error
#define rvm_error(log, msg) \
if (log.start_msg(log.FAILURE_TYP, log.ERROR_SEV)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_fatal
#define rvm_fatal(log, msg) \
if (log.start_msg(log.FAILURE_TYP, log.FATAL_SEV)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_trace
#define rvm_trace(log, msg) \
if (log.start_msg(log.DEBUG_TYP, log.TRACE_SEV)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_debug
#define rvm_debug(log, msg) \
if (log.start_msg(log.DEBUG_TYP, log.DEBUG_SEV)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_verbose
#define rvm_verbose(log, msg) \
if (log.start_msg(log.DEBUG_TYP, log.VERBOSE_SEV)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

```

```

#undef rvm_note
#define rvm_note(log, msg) \
if (log.start_msg(log.NOTE_TYP)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_report
#define rvm_report(log, msg) \
if (log.start_msg(log.REPORT_TYP)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_command
#define rvm_command(log, msg) \
if (log.start_msg(log.COMMAND_TYP)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_protocol
#define rvm_protocol(log, msg) \
if (log.start_msg(log.PROTOCOL_TYP)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_transaction
#define rvm_transaction(log, msg) \
if (log.start_msg(log.TRANSACTION_TYP)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

#undef rvm_cycle
#define rvm_cycle(log, msg) \
if (log.start_msg(log.CYCLE_TYP)) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void

```

```
#undef rvm_user
#define rvm_user(n, log, msg) \
if (log.start_msg(log.USER_TYP[n])) { \
    void = log.text(psprintf("\'%s\'", %0d: %s", __FILE__, __LINE__, msg)); \
    log.end_msg(); \
} \
void = void
```